MANNASEED

Hidden Provision. Eternal Word.

A Censorship-Resistant System for Preserving
&
Distributing the King James Bible

TECHNICAL WHITEPAPER

Version 1.4

April 2026

Open Source · MIT Licensed · No Network Required

github.com/PatriotCrypto/MannaSeed

blockchainbible.org

Abstract

MannaSeed is an open-source, two-component system designed to ensure that the complete King James Version of the Bible remains permanently available and discreetly accessible regardless of legal, political, or technological pressure to suppress it. The first component is a permanence layer: all 66 books of the KJV have been inscribed as 66 individual Bitcoin Ordinal inscriptions on the Bitcoin mainnet, producing an immutable and globally replicated copy of the text that cannot be deleted, modified, or selectively censored. The second component is an access layer: a single self-contained HTML file that functions visibly as a calculator while concealing an AES-256-GCM encrypted copy of the Bible behind a formula-based PIN gate, BIP39 seed-phrase authentication, and an editable journal that serves as a decoy. The application performs no network requests, decrypts content into RAM only, and wipes its state on tab change, idle timeout, or explicit lock. This whitepaper documents the system's threat model, cryptographic design, implementation, current status, and known limitations.

Contents

Abstract 2
1. Introduction 4
2. Threat Model 4
3. System Architecture 5
4. Permanence Layer: Bitcoin Ordinal Inscriptions 6
5. Access Layer: The MannaSeed Application 6
6. Cryptographic Design 7
7. Authentication and Decoy System 9
8. Defensive Mechanisms 10
9. Implementation Status 10
10. Security Analysis and Limitations 11
11. Distribution Model 12
12. Roadmap 13
13. Conclusion 13
References 14
Appendix A: File Hashes 15
Appendix B: Inscription Manifest 16
Appendix C: Glossary 17
Colophon 18

1. Introduction

1.1 Motivation

Sacred texts have historically been targets of suppression. Throughout the past two millennia, the Bible has been banned, burned, and outlawed in many jurisdictions; in several modern countries it remains contraband or its possession carries severe penalties. The general trend in centralized digital infrastructure — cloud-hosted applications, app-store gatekeeping, content scanning on consumer devices — means that a single policy decision by a small number of platform operators could remove the dominant Bible reading applications from circulation in any jurisdiction within days.

This whitepaper does not assert that such suppression is imminent in any specific country. It addresses the engineering question: if it were to occur, what technical infrastructure would need to exist beforehand to preserve the text and make it discreetly available? Building such infrastructure in advance is significantly easier and less risky than building it under pressure.

1.2 Design Goals

MannaSeed is designed around four primary goals:

- Permanence: a publicly verifiable, immutable copy of the complete KJV that cannot be removed by any single party.
- Discretion: an access tool whose presence on a device does not reveal that the Bible is present.
- Independence: complete operation without network connectivity, third-party services, or accounts.
- Distributability: small enough and self-contained enough to spread freely through any digital medium, including offline media.

1.3 Scope

This document covers the system architecture, cryptographic design, threat model, current implementation status, and known limitations. It does not provide step-by-step deployment instructions for end users — those are maintained separately in the project repository. It also does not constitute legal or operational advice for users in restrictive jurisdictions; users in such contexts should consult appropriate local resources.

2. Threat Model

2.1 Adversaries

The threat model considers three classes of adversary, in increasing order of capability:

Class A — Automated content scanning. Software, typically deployed by platform operators, network providers, or device manufacturers, that scans device storage or network traffic for known content signatures (such as text strings or file hashes corresponding to canonical Bible passages). This adversary cannot interact with the user; it relies on pattern matching against accessible data.

Class B — Compelled inspection. A human inspector with physical access to an unlocked device who reviews installed applications and visible files. This adversary can open applications and observe their behavior but does not have specialized forensic tools or extended time.

Class C — Forensic extraction. A trained operator using commercial mobile forensic platforms (for example, devices and software offered by major vendors in this category) who has extended physical access to the device and can perform full filesystem extraction, recover deleted data, and examine browser storage.

2.2 Protected Properties

Against Class A adversaries, the system protects:

- Bible content not appearing in plaintext on disk.
- Application files not matching known Bible-application signatures.
- No characteristic network traffic that could be used to identify the application or its content.

Against Class B adversaries, the system protects:

- The application appearing and functioning as an ordinary calculator.
- A plausible secondary surface (an editable journal) that opens when an incorrect access phrase is entered, without any indication that an incorrect entry was made.

Against Class C adversaries, the system protects:

- Bible content remaining encrypted at rest under AES-256-GCM with a key derived from a user-controlled secret.
- Application code not containing recognizable Bible text strings or Bible-related identifiers.
- Decrypted content existing only in RAM during active reading sessions, with multiple mechanisms for prompt RAM wipe.

2.3 Out-of-Scope Threats

The following are explicitly outside the scope of what MannaSeed defends against:

- A Class C adversary who has obtained both the device and the user's seed phrase. The system provides confidentiality through the secret; if the secret is compromised, the content is recoverable.
- A Class C adversary using video surveillance, keylogging hardware, or compromised device firmware. The system assumes the device is not actively monitoring the user during legitimate access sessions.
- Coercion past the point of seed-phrase disclosure. The decoy journal provides plausible cover for a single demonstration; it does not guarantee the user can withhold information indefinitely under sustained coercion. (See §10.4.)
- Targeted attacks against the user's identity, social network, or cryptographic key generation entropy on a compromised device.

3. System Architecture

MannaSeed consists of two independent layers:

Permanence layer. A copy of the complete KJV inscribed onto the Bitcoin blockchain. This layer makes the text impossible to remove. It does not provide privacy or discretion — anyone can fetch the inscriptions from any Bitcoin node — but it ensures the text continues to exist regardless of any party's actions.

Access layer. A self-contained application (currently a single HTML file; native iOS and Android implementations planned) that allows a user to read the Bible privately on a personal device. The access layer ships a pre-encrypted copy of the Bible bundled within the application; it does not download the text from the blockchain. The blockchain copy serves as the canonical reference and as a recovery mechanism if the application is ever lost or destroyed.

The two layers are independent. The permanence layer can serve any compatible reader; the access layer can be re-distributed without any blockchain interaction. This separation ensures that compromise or unavailability of one does not compromise the other.

4. Permanence Layer: Bitcoin Ordinal Inscriptions

4.1 Rationale for Bitcoin

Among publicly accessible distributed systems, Bitcoin's base layer offers the highest known degree of censorship resistance and data permanence:

- The chain is replicated across tens of thousands of independent nodes globally.
- The cost to alter or remove confirmed historical data is the cumulative proof-of-work securing all subsequent blocks, currently representing a hash rate that no known adversary can practically reverse.
- Inscriptions, once confirmed, are part of the witness data of standard transactions and are mirrored by every full node that retains witness data.
- No central administrator exists who could be compelled to remove the data.

4.2 Inscription Strategy

The complete KJV (4,330,220 bytes of plaintext) was inscribed as 66 individual inscriptions, one per book, rather than as a single monolithic inscription or as two inscriptions for the Old and New Testaments. This design provides several advantages:

- On-demand retrieval: a reader interested in a single book can fetch only that book.
- Resilience: damage or unavailability of one inscription does not affect the others.
- Verifiability: each book's SHA-256 hash can be computed and compared independently against an external manifest.
- Granularity for cataloging and indexing by external services.

4.3 Inscription Process

Inscriptions were created using the Ordinals protocol over Taproot witness data. The full process is documented in the project repository. The summary is:

- Each book's plaintext was prepared as a single .txt file in canonical form (verse-numbered prose).
- Each file was committed as a Taproot script-path inscription.
- Reveal transactions were broadcast at low fee rates over a multi-day period in April 2026.
- Each resulting inscription transaction ID was recorded in a public manifest.
- A two-phase verification was performed: (1) the original source file's SHA-256 was compared against the canonical KJV reference, and (2) each inscription's on-chain content was re-fetched from a public Ordinals indexer and its SHA-256 compared against the manifest.

4.4 Verification

All 66 inscriptions passed both verification phases. The inscription manifest is published at:

`github.com/PatriotCrypto/MannaSeed/blob/main/INSCRIPTIONS.csv`

The manifest contains, for each of the 66 books, the book number, name, file SHA-256, inscription transaction ID, and inscription number assigned by the Ordinals protocol.

5. Access Layer: The MannaSeed Application

5.1 Form Factor

The first delivered implementation of the access layer is a single HTML file. This form was chosen for the following reasons:

- It runs in any modern browser on any operating system, with no installation step.
- It can be transferred through any channel that handles files: email attachments, USB drives, physical media, peer-to-peer networks.
- It is small enough (under 2 MB) to fit comfortably on every realistic distribution medium.
- It cannot be silently revoked from a centralized application store.
- Users can verify the file's integrity by comparing its SHA-256 against the published value before opening it.

Native iOS and Android applications are planned (see §12) to address the user-experience limitations of file-based distribution on mobile devices.

5.2 Calculator Surface

When opened, the application displays a fully functional calculator. It supports the standard four arithmetic operations, decimal entry, sign change, percentage, and chained operations. Both touch input and physical keyboard input are supported. The calculator behaves identically to a standard mobile calculator application; it does not exhibit any hidden affordances that would distinguish it from one.

The hidden interface is reached through a user-defined formula — an arithmetic expression of at least three keystrokes terminating in the equals key — which the user creates during initial setup. When the calculator detects that the keystroke sequence terminating in the equals key matches the stored formula, it transitions to the seed-phrase entry screen instead of displaying a result.

5.3 Single-File Constraint

The application is a single self-contained file. It contains:

- The HTML structure and CSS styling.
- The complete BIP39 English wordlist (2,048 words).
- The encrypted Bible blob (gzipped plaintext encrypted under AES-256-GCM, base64-encoded).
- All JavaScript code for the calculator, setup flow, decryption, reader, and journal.

It loads no external resources. A Content Security Policy header is set in the document to formally prohibit any external resource loading even if the document were modified by an intermediary.

6. Cryptographic Design

6.1 Primitives

All cryptographic operations use the W3C Web Crypto API, which exposes platform-native cryptographic primitives in modern browsers. The application uses the following primitives:

Symmetric encryption	AES-256-GCM (authenticated encryption)
Key derivation	PBKDF2-HMAC-SHA-256, 100,000 iterations, 16-byte salt
Hashing	SHA-256 (used for PIN hashing and BIP39 checksum)
Random number generation	crypto.getRandomValues (CSPRNG)
Mnemonic encoding	BIP39 with the official English wordlist
Compression	gzip (via DecompressionStream)

6.2 Two-Layer Key Wrapping

A direct approach — encrypting the Bible with a key derived from the user's seed phrase — would require either (a) every distributed copy to use the same seed phrase, defeating the purpose, or (b) the user to re-encrypt the entire Bible blob during setup, an operation requiring several seconds of compute and producing a re-encrypted file that complicates verification.

The system instead uses a two-layer key wrapping construction:

- **Layer 1 (build time).** A 256-bit master key is generated. The Bible plaintext is gzipped, encrypted with AES-256-GCM under the master key, base64-encoded, and embedded in the HTML file. The same encrypted blob and the same master key ship with every copy of the application.

- **Layer 2 (first run).** The user generates a 12-word BIP39 mnemonic on their device. A random 16-byte salt is generated. A wrapping key is derived from the mnemonic (and optional passphrase) using PBKDF2 with the salt. The master key is encrypted ("wrapped") under the wrapping key with a fresh 12-byte IV. The wrapped master key, salt, and IV are stored in the browser's localStorage.

At unlock time, the wrapping key is re-derived from the user's mnemonic input plus the stored salt. If the input is correct, the wrapping key successfully unwraps the master key, and the master key decrypts the bundled Bible blob in RAM. If the input is incorrect, the unwrap operation fails (AES-GCM authenticates the ciphertext) and the application opens the journal silently.

6.3 Mnemonic Generation

Mnemonics are generated according to the BIP39 specification:

- 128 bits of entropy are obtained from crypto.getRandomValues.
- A 4-bit checksum is computed as the high 4 bits of SHA-256(entropy).
- The 132-bit string (entropy || checksum) is split into twelve 11-bit groups.
- Each group indexes a word from the BIP39 English wordlist (2,048 words).

The resulting mnemonic is BIP39-conformant and compatible with standard cryptocurrency wallet software. This is intentional: it allows interested users to verify the mnemonic's validity using independent tools and provides assurance that the entropy was correctly handled.

6.4 Formula PIN

The formula PIN is the keystroke sequence (digits, operators, and the equals key) entered on the visible calculator. During setup, the sequence is canonicalized as a string, hashed with SHA-256, and the hash stored in localStorage. The plaintext sequence is never stored. At runtime, every equals-key press triggers a comparison: the current keystroke log is hashed and compared against the stored hash; if they match, the seed-phrase entry screen opens; if they do not, the calculator computes the arithmetic result normally.

The PIN is not a cryptographic protection — it serves only to gate the appearance of the seed-phrase entry screen. It must be at least three keystrokes long.

6.5 Two Security Modes

Users select between two security modes during setup:

Basic mode. The 12-word mnemonic alone is the unlock secret. Entering the correct 12 words opens the Bible. Entering any other 12 words opens the journal.

Advanced mode. The unlock secret is the 12-word mnemonic concatenated with a user-chosen passphrase (separated by a delimiter). The 12 words alone open the journal. The 12 words plus the correct passphrase open the Bible. The passphrase is never stored anywhere on the device; it exists only in the user's memory and is entered on each unlock.

Advanced mode provides plausible deniability against a Class B or Class C adversary who obtains the written-down mnemonic: the mnemonic, when entered, opens the journal, which contains plausible content. Without knowledge that a passphrase exists, an inspector has no reason to suspect that anything else is concealed.

7. Authentication and Decoy System

7.1 Setup Flow

On first launch the application detects no setup state in localStorage and presents the setup flow:

1. Formula PIN creation. The user enters an arithmetic sequence on the keypad. The sequence is hashed and stored.

2. Security mode selection. The user selects basic or advanced.

3. Passphrase (advanced only). The user enters and confirms a passphrase of at least six characters.

4. Mnemonic generation and display. A 12-word BIP39 mnemonic is generated and displayed for the user to write down.

5. Mnemonic verification. The user re-enters all 12 words in order. Autocomplete activates after four letters. A back button allows correction. All 12 must be correct to proceed.

6. Master key wrapping and storage. The wrapping key is derived; the master key is wrapped; the wrapped key, salt, and IV are written to localStorage.

7. Journal initialization. The decoy journal is populated with four starter entries (everyday topics).

After setup, all sensitive in-memory state is zeroed, and the application returns to the calculator surface.

7.2 Unlock Flow

On any subsequent launch, the application detects setup state and shows the calculator. The user enters the formula PIN; the equals-key handler matches the hash, and the seed-phrase entry screen appears. The user enters all 12 words. In advanced mode, a passphrase field appears after the 12th word, and the user enters the passphrase. The unlock secret is constructed; the wrapping key is derived; the master key is unwrapped.

If unwrap succeeds, the bundled Bible blob is decrypted, unzipped, parsed by book, and the reader opens. If unwrap fails (AES-GCM authentication failure due to wrong key), control routes silently to the journal. From the perspective of an observer, both paths look identical — a brief loading indicator followed by the appearance of either the Bible reader or the journal.

7.3 Decoy Journal

The journal is a fully functional notes application. It is initialized on first run with four starter entries (groceries, household repairs, vacation planning, gift ideas) chosen to be unremarkable on inspection. The user can read, edit, create, and delete entries; entries persist in localStorage. Adding genuine personal notes is encouraged: the more authentic the journal's content, the more credible the cover.

Importantly, the journal does not display any indicator that it was reached via decoy routing. The journal opens identically whether the user explicitly enters the wrong phrase to access it or whether the application routed there after a failed unlock attempt.

8. Defensive Mechanisms

8.1 RAM-Only Decryption

Decrypted Bible content is held in JavaScript variables only. It is never written to localStorage, sessionStorage, IndexedDB, the file system, or any other persistent storage. Closing the browser tab discards the variables; switching applications, on most platforms, eventually causes the browser process to be paused and its working memory to be released.

8.2 Kill Switch

Three independent mechanisms wipe in-memory Bible state and return the application to the calculator surface:

- Manual lock: every Bible reader and journal screen displays a visible lock button that triggers immediate wipe.

- Page Visibility API: when the browser fires a visibility change event indicating the tab is no longer visible (tab switch, application switch, screen lock), the application immediately wipes if the Bible reader or journal is active.
- Idle timeout: a 60-second timer is reset on user interaction. If it expires, the application wipes.

In all three cases, the wipe sets relevant variables to null or empty, clears any pending timers, and renders the calculator surface as if the application had just been opened.

8.3 Content Security Policy

The HTML document declares a strict Content Security Policy that prohibits all external resource loading and all network connections. This serves two purposes: (a) it prevents accidental network traffic that could be observed by a network monitor, and (b) it provides a check against tampering: an inspector who modifies the file to attempt to exfiltrate data will trigger the CSP and the modification will fail to function.

8.4 Forensic Surface

When the application is at rest (no active session), a forensic examination of the device finds:

- A single HTML file that opens to a calculator UI.
- Browser localStorage entries containing the formula PIN hash, the wrapped master key, a salt, an IV, a method indicator, and the journal contents.
- No plaintext Bible content anywhere.
- No identifiable Bible-related strings in the HTML or JavaScript.

The localStorage entries are recognizable as cryptographic data — base64-encoded byte strings — but contain no information about what they protect. Without the user's mnemonic (and passphrase, in advanced mode), they cannot be decrypted.

9. Implementation Status

9.1 Phase 0 — Permanence Layer

Status: Complete. All 66 books of the KJV were inscribed onto the Bitcoin mainnet between April 22 and April 25, 2026, using the Ordinals protocol. All 66 inscription transaction IDs are published in the project manifest and have been independently verified via SHA-256 comparison between the source files and the on-chain content. The total fee cost was approximately 1.42 million satoshis.

9.2 Phase 1 — Access Layer (HTML)

Status: Complete. The first public release of the HTML access application is published as v1.0.0 in the project repository. File details:

Filename	mannaseed.html
Size	1,887,707 bytes (1.89 MB)
SHA-256	e42b5793729922a85cc479bbded28ad04276fb270a51899c49eb5462b2fbf617
Distribution	GitHub Releases (github.com/PatriotCrypto/MannaSeed/releases)

The application has been functionally tested against all flows described in this document (setup, unlock, decoy routing, kill switch, idle timeout, edit-and-save journal entries, book navigation across all 66 books).

9.3 Phase 2 and Beyond

Phases 2 (iOS native), 3 (Android native), and 4 (additional translations) are planned but not yet implemented. See §12 for the roadmap.

10. Security Analysis and Limitations

10.1 Symbolic Encryption Caveat

The master key used to encrypt the bundled Bible blob is embedded in the HTML file in plaintext (as a hexadecimal constant). This is intentional: every distributed copy of the application uses the same encrypted blob and the same master key. The user-specific protection is the wrapped master key in localStorage, which is bound to the user's mnemonic.

A consequence is that any party who possesses a copy of the HTML file can extract the master key and decrypt the bundled Bible blob, recovering the plaintext. The encryption of the bundled blob is therefore not a confidentiality measure against parties who hold the file — it is a measure that prevents the file's contents from being recognizable to automated content scanning or string-based forensic search. This distinction is important and is explicitly acknowledged here.

The user-specific cryptographic protection — the AES-GCM wrapping of the master key under a PBKDF2-derived key from the user's mnemonic — is real. Without the user's mnemonic, the wrapped master key in their localStorage cannot be unwrapped.

10.2 PBKDF2 Iteration Count

PBKDF2 with 100,000 iterations of SHA-256 is on the lower end of currently recommended values (the OWASP recommendation as of 2023 is 600,000 for PBKDF2-SHA-256). The current value was chosen to keep unlock latency on slower mobile devices below approximately 0.5 seconds. Future versions should consider increasing this value or migrating to Argon2id, which is more resistant to GPU-based attacks. This is identified as a near-term improvement.

10.3 localStorage Forensic Visibility

Browser localStorage entries are stored on the device's file system and are recoverable by Class C forensic tools. The entries themselves do not reveal their purpose, but their presence indicates that some browser-based application has stored data. A determined examiner who finds the localStorage entries and the HTML file in proximity may infer that they are related, even if neither alone conveys this. There is no current mitigation for this beyond user awareness; users in high-risk environments should consider running the application in a private/incognito browser window, which does not persist localStorage and therefore requires re-running setup on every session.

10.4 Coercion Resistance Limits

The decoy system is designed to provide a single plausible demonstration. It does not protect against an adversary who is aware of MannaSeed's existence and the possibility of a passphrase, who can compel the user to disclose the passphrase, or who is willing to coerce the user past the point of plausible compliance. Users facing these threat levels require operational security measures beyond what software can provide.

10.5 Mobile File Distribution Limitations

On mobile platforms, opening a downloaded HTML file from a Files application introduces several user-experience and forensic compromises:

- The file appears in the device's Downloads folder with its filename visible.
- The browser address bar displays a file:// URL.
- The application cannot be added to the home screen as an app icon.

These compromises do not affect cryptographic security but reduce the credibility of the calculator disguise on mobile devices. Native iOS and Android applications (Phases 2 and 3) will address these issues by appearing in the device's normal application drawer alongside other utilities.

10.6 Pending Independent Audit

The cryptographic design and implementation have not yet undergone independent third-party security review. A formal audit is identified as a prerequisite for promoting MannaSeed for use in adversarial environments. Users in such environments should consider the system experimental until an audit has been completed.

11. Distribution Model

11.1 Open Source, MIT Licensed

The complete source code, build scripts, and documentation are released under the MIT License. The project is permanently free; there is no commercial component.

11.2 Channels

The application is distributed through:

- GitHub Releases (primary channel, with published SHA-256 for verification).
- blockchainbible.org (web download with the same SHA-256 published).
- Direct file transfer (encouraged): the file is small enough to share by email, messaging applications that allow file attachments, USB drive, or any other medium that handles arbitrary files.

11.3 Verification

Users are encouraged to verify any received copy of the application against the published SHA-256 before running it. On macOS and Linux:

```
shasum -a 256 mannaseed.html
```

On Windows:

```
certutil -hashfile mannaseed.html SHA256
```

The output should match the SHA-256 published in this document and on the project release page.

12. Roadmap

Phase 0 (complete). Inscribe complete KJV onto Bitcoin mainnet as 66 individual Ordinal inscriptions.

Phase 1 (complete). Single-file HTML access application with calculator disguise, AES-256-GCM encrypted Bible, BIP39 authentication, decoy journal, and kill switch.

Phase 2 (planned). Native iOS application that appears in the device's application drawer as a calculator. Includes the same cryptographic architecture as the HTML version, plus iOS-specific defensive mechanisms (Secure Enclave key storage, biometric unlock, screenshot suppression).

Phase 3 (planned). Native Android application with equivalent functionality and Android-specific defensive mechanisms (Keystore key storage, biometric unlock, FLAG_SECURE).

Phase 4 (planned). Additional public-domain Bible translations (e.g., World English Bible, Berean Standard Bible) inscribed onto the blockchain and selectable in the application.

Phase 5 (planned). Translations into other languages, beginning with those for which a complete public-domain or openly licensed translation exists.

Ongoing. Independent cryptographic audit; PBKDF2 iteration count increase or migration to Argon2id; user-experience refinement based on field feedback.

13. Conclusion

MannaSeed is a working system, not a proposal. The permanence layer is on chain and verifiable today. The access layer is published and downloadable today. The cryptographic design is conventional, the implementation is auditable, and the limitations are openly documented.

The system's purpose is preparation rather than response. Building censorship-resistant infrastructure is significantly easier when no one is yet trying to suppress it. The project's goal is to make the King James Bible so widely distributed and so technically resilient that no single party — government, platform operator, or future technology change — can practically remove it from circulation.

References

[1] S. Nakamoto. "Bitcoin: A Peer-to-Peer Electronic Cash System." 2008. bitcoin.org/bitcoin.pdf

[2] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe. "BIP-0039: Mnemonic code for generating deterministic keys." 2013. github.com/bitcoin/bips/blob/master/bip-0039.mediawiki

[3] C. Rodarmor. "Ordinals Theory Handbook." docs.ordinals.com

[4] M. Dworkin. "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC." NIST Special Publication 800-38D, 2007.

[5] B. Kaliski. "PKCS #5: Password-Based Cryptography Specification Version 2.0." RFC 2898, 2000.

[6] W3C. "Web Cryptography API." W3C Recommendation, 2017. w3.org/TR/WebCryptoAPI/

[7] W3C. "Page Visibility Level 2." W3C Recommendation, 2022. w3.org/TR/page-visibility/

[8] W3C. "Content Security Policy Level 3." W3C Working Draft, 2023. w3.org/TR/CSP3/

[9] OWASP Foundation. "Password Storage Cheat Sheet." cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[10] The King James Version of the Holy Bible. Public domain in the United States.

Appendix A: File Hashes

The current canonical release artifacts and their cryptographic hashes:

mannaseed.html (v1.0.0)	e42b5793729922a85cc479bbded28ad04276fb270a51899c49eb5462b2fbf617
Size	1,887,707 bytes (1.89 MB)
Inscription manifest	See INSCRIPTIONS.csv in project repository

Users should verify any received copy against these published hashes before use. A SHA-256 mismatch indicates a tampered or corrupted copy, and the file should not be opened.

Appendix B: Inscription Manifest

The complete list of 66 inscription transaction IDs, with their corresponding book numbers, names, and content SHA-256 values, is maintained in the project repository at:

```
github.com/PatriotCrypto/MannaSeed/blob/main/INSCRIPTIONS.csv
```

Each inscription can be independently verified by fetching its content from any Bitcoin Ordinals indexer (e.g., ordinals.com) using the published transaction ID and computing the SHA-256 of the returned content.

Appendix C: Glossary

AES-256-GCM. Advanced Encryption Standard with a 256-bit key in Galois/Counter Mode. Provides both confidentiality and authentication; decryption fails detectably if the ciphertext or the key is incorrect.

BIP39. Bitcoin Improvement Proposal 39, defining a standard method for converting random entropy into a sequence of words drawn from a fixed wordlist, designed to be readable and re-typable by humans.

CSP. Content Security Policy. A browser security mechanism, declared via HTTP header or meta tag, that restricts what resources a document may load and what behaviors it may exhibit.

KJV. King James Version. An English translation of the Bible first published in 1611, in the public domain in the United States.

localStorage. A browser-provided key-value storage mechanism that persists data per origin across browser sessions.

Ordinals. A protocol for inscribing arbitrary content onto Bitcoin satoshis as part of Taproot witness data, producing a permanent on-chain record of the content.

PBKDF2. Password-Based Key Derivation Function 2. A standard algorithm for deriving a cryptographic key from a low-entropy secret (such as a password) by applying a hash function many times, intended to slow down brute-force attempts.

Taproot. A Bitcoin protocol upgrade activated in November 2021 that introduced new transaction structures, including the witness data format used by the Ordinals protocol.

Colophon

This whitepaper documents MannaSeed as of April 2026. The MannaSeed project is open source, MIT licensed and maintained on GitHub. Inquiries, contributions, and audit requests should be directed to the project repository.

The project is, and will always remain, free. There is no commercial component, no premium tier, and no advertising.

The MannaSeed application and this whitepaper were built using Claude Opus 4.7 by Anthropic.

Repository: github.com/PatriotCrypto/MannaSeed

Web: blockchainbible.org

License: MIT